AIMS 5771 Advanced Topics in Artificial Intelligence II
Homework 1

General Guidelines:

- Please check the submission deadline on **Blackboard** and submit your solutions via **Blackboard**.
- Each homework's deadline has a grace period of 12 hours.
- **Do not** close your browser of app before you have successfully uploaded your files. It is your own responsibility of keeping your file integrity.

1. (50 points) **Hardware-Aware Attention: The FlashAttention Tiling Primitive & Online Softmax**

   Standard scaled dot-product attention computes:

   $$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

   In standard implementations, materializing the intermediate $N \times N$ attention matrix on High Bandwidth Memory (HBM) creates an $\mathcal{O}(N^2)$ memory bandwidth bottleneck, making the layer memory-bound rather than compute-bound. FlashAttention circumvents this by computing exact attention block-by-block within on-chip SRAM using an **online softmax** formulation.

   (a) (15 points) **GPU Memory Hierarchy and the Memory Wall:**

   i. Contrast the physical storage capacities and theoretical access bandwidths of GPU High Bandwidth Memory (HBM, e.g., 40–80 GB) versus on-chip Static RAM (SRAM, e.g., ∼20 MB).

   ii. Explain why standard attention execution is bottlenecked by memory I/O rather than arithmetic throughput (FLOPs) during long-context training and inference. Trace the memory reads and writes of intermediate matrices $S = QK^\top$ and $P = \text{softmax}(S)$ between HBM and SRAM.

   (b) (20 points) **Mathematical Derivation of Online Softmax:** Let a row vector $x \in \mathbb{R}^{2B}$ be partitioned into two contiguous sub-blocks $x^{(1)}, x^{(2)} \in \mathbb{R}^B$. For any vector v, define its maximum as $m(v) = \max_i v_i$, its unnormalized exponentiated vector as $f(v) = e^{v - m(v)}$, and its scalar normalizing denominator as $\ell(v) = \sum_i e^{v_i - m(v)}$.

   i. Define the global maximum of the combined vector $x = [x^{(1)}, x^{(2)}]$ as $m(x) = \max(m(x^{(1)}), m(x^{(2)}))$. Prove algebraically that the combined denominator $\ell(x)$ can be computed strictly from the summary statistics $(m(x^{(1)}), \ell(x^{(1)}))$ and $(m(x^{(2)}), \ell(x^{(2)}))$ without ever having all $2B$ unscaled elements in SRAM simultaneously. Show the exact algebraic transition rule for $\ell(x)$.

   ii. Express the combined unnormalized vector $f(x)$ in terms of $f(x^{(1)})$ and $f(x^{(2)})$, and prove that $\frac{f(x)}{\ell(x)}$ is mathematically identical to the standard vector $\text{softmax}(x)$.

   (c) (15 points) **Backward Pass Dynamics and SRAM Recomputation:**

   i. In standard backpropagation, why does computing gradients through the attention sub-layer require $\mathcal{O}(N^2)$ memory allocation?

   ii. Explain how FlashAttention reduces the backward pass activation memory footprint to $\mathcal{O}(N)$.

   iii. Explain the hardware-level reason why recomputing $QK^\top$ dynamically in SRAM during backpropagation achieves a wall-clock speedup compared to reading the cached $N \times N$ matrix from HBM.

2. (50 points) **KV Cache Compression Paradigms: From MQA/GQA to Latent Weight Absorption (MLA)**

   Consider a modern decoder-only language model executing autoregressive generation at batch size B, sequence length N, hidden state dimension d_{model}, head count n_h, and per-head dimension d_h, such that $d_{\text{model}} = n_h \cdot d_h$.

(a) (20 points) **Analytical Cache Footprint and Decoding Bottlenecks:**

i. Derive the analytical KV cache memory footprint (in bytes, assuming 16-bit half-precision floating-point representations) required per generated token across each of the following architectures:

1. Standard Multi-Head Attention (MHA)
2. Multi-Query Attention (MQA)
3. Grouped-Query Attention (GQA) with G query groups ($1 < G < n_h$)

ii. Explain why reducing the KV cache size drastically reduces per-token decoding latency in autoregressive generation even though the total theoretical floating-point operations (FLOPs) required remain practically unchanged.

(b) (30 points) **Multi-Head Latent Attention (MLA) and Weight Absorption:** DeepSeek's Multi-Head Latent Attention (MLA) compresses the key-value representation into a low-dimensional latent vector $c_t^{KV} \in \mathbb{R}^{1\times d_c}$ where $d_c \ll n_h \cdot d_h$:

$$c_t^{KV} = h_t W^{DKV}, \quad K_t = c_t^{KV} W^{UK}, \quad V_t = c_t^{KV} W^{UV}$$

where $W^{DKV} \in \mathbb{R}^{d_{\text{model}}\times d_c}$ is the down-projection matrix, and $W^{UK}, W^{UV} \in \mathbb{R}^{d_c\times(n_h d_h)}$ are the up-projection matrices. For each head $i \in \{1, \ldots, n_h\}$, the corresponding slice of the key up-projection matrix is $W_i^{UK} \in \mathbb{R}^{d_c\times d_h}$, and the generated per-head query is $Q_{t,i} \in \mathbb{R}^{1\times d_h}$.

i. (15 points) During autoregressive generation, caching uncompressed keys K_t and values V_t destroys the memory benefits of latent compression. Using the associativity of matrix multiplication, prove that the key up-projection matrix W_i^{UK} can be absorbed directly into the per-head query vector $Q_{t,i}$ prior to computing attention, formally deriving the absorbed query representation:

$$\tilde{Q}_{t,i} = Q_{t,i}(W_i^{UK})^\top \in \mathbb{R}^{1\times d_c}$$

ii. (15 points) Using the absorbed query representation $\tilde{Q}_{t,i} = Q_{t,i}(W_i^{UK})^\top$, write out the reformulated attention logit equation $S_{t,j,i}$ between token t and cached historical token j directly against the compressed latent vector $c_j^{KV} \in \mathbb{R}^{1\times d_c}$.